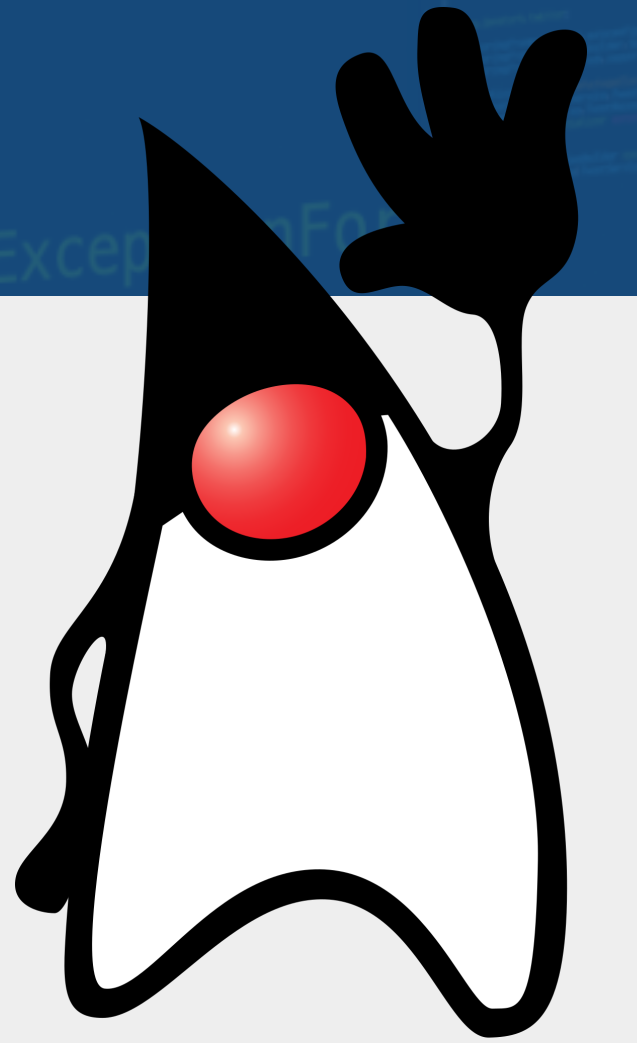


# Java Programlama Dili

## 5. Bölüm

# Java'nın Tip Sistemi

Akın Kaldıroğlu



# Amaç



- Bu bölümde tiplere ve değişkenlere giriş yapılacaktır ve Java'nın tip sistemi ele alınacaktır

# Konular



- **Veri, Tip ve Değişken**
  - Tipli ve Tipsiz Diller
- **Java'nın Tipleri**
- **Ödevler**
- **Bundan Sonra**

# Veri, Tip ve Değişken



# Veri (Data) - I



- Programlama dillerinde farklı türlerde veriler (data) vardır:
  - `5`
  - `a`
  - `Selam`
  - `3.14`
  - `3.141592653589793`
  - `true`
  - `false`
  - `{1, 2, 3, 4, 5}`
  - `1, Semih, Telli, Satış, true`

# Veri (Data) - II



- `5` bir tam sayıdır (integer)
- `a` bir karakterdir (character)
- `Se1am` bir karakter dizisidir (character sequence), metin (text) parçasıdır
- `3.14` ve `3.141592653589793` virgüllü sayıdır (floating-point number) ama ikincisinde çok daha fazla detay vardır
- `true` ve `false` mantıksal (logical) birer değerdir, bir önermenin doğruluk değerini gösterir

# Veri (Data) - III



- `{1, 2, 3, 4, 5}` beş sayıdan oluşan bir dizidir (array)
- `1, Semih, Telli, Satış, true`, örneğin bir çalışanı (employee) temsil eden ve beş ayrı veriden oluşan bir nesnedir (object)

# Veri (Data) - IV



- Tüm veriler bir değere (value) sahiptir ve bu değer değişmez
- Veri, ister 5 gibi tek parça olsun ister {1, 2, 3, 4, 5} gibi beş sayıdan oluşsun farketmez, bir değeri temsil eder ve bu değer değişmez
  - dolayısıyla veriler aynı zamanda birer **sabitedir (constant)**
- Veriler farklı yapıya, özelliklere hatta kısıtlara sahiptir
- Verilerin sahip olduğu yapı, özellik ve kısıtların tümüne, tip (type) adı verilir

# Veri (Data) - V



- Burada *veri (data)*, *veri değeri (data value)* ve *değer (value)* aynı anlama gelir; dolayısıyla bu kelimeleri birbirleri yerine kullanabiliriz

# Değişken (Variable) - I



- Programlama dillerinde verilerin değerlerini (data values) tutan yapılara **değişken (variable)** denir
- Veri sabittir ama onu gösteren ya da onu tutan şey değişebilir
  - Değişkenler, verilerin içine konduğu bir kutu (box) gibi düşünülebilir
  - Bir kutuya farklı zamanlarda farklı veriler konabilir
- Veriler de onları tutan, gösteren değişkenler de bellekte bulunur, orada yaşarlar

# Değişken (Variable) - II



- Değişkenlere değer atanır, üzerlerinde işlemcilerle (operators) işlemler yapılır ve bunun sonucunda yeni değerlere sahip olur
- Değişkenler metotlara geçilip, metotlardan geri döndürülerek veriler program içinde gezdirilir, farklı program parçaları arasında paylaşılır ve bu yolla da değişkenler yeni değerlere sahip olur

# Değişken (Variable) - III



- Yani değişkenlerin değeri devamlı değişir, bu yüzden onlara **değişken (variable)** denir
- **Değişken, değeri değişen şeydir (variable is something whose value varies)**
- Dolayısıyla değişken, farklı zamanlarda farklı değerleri göstermeye yeteneğine sahip olan bir programlama soyutlamasıdır (abstraction)
- Bu anlamda değişken, matematikteki  $x$  ya da  $y$  gibi bir soyutlamadır

# Tip (Type) - I



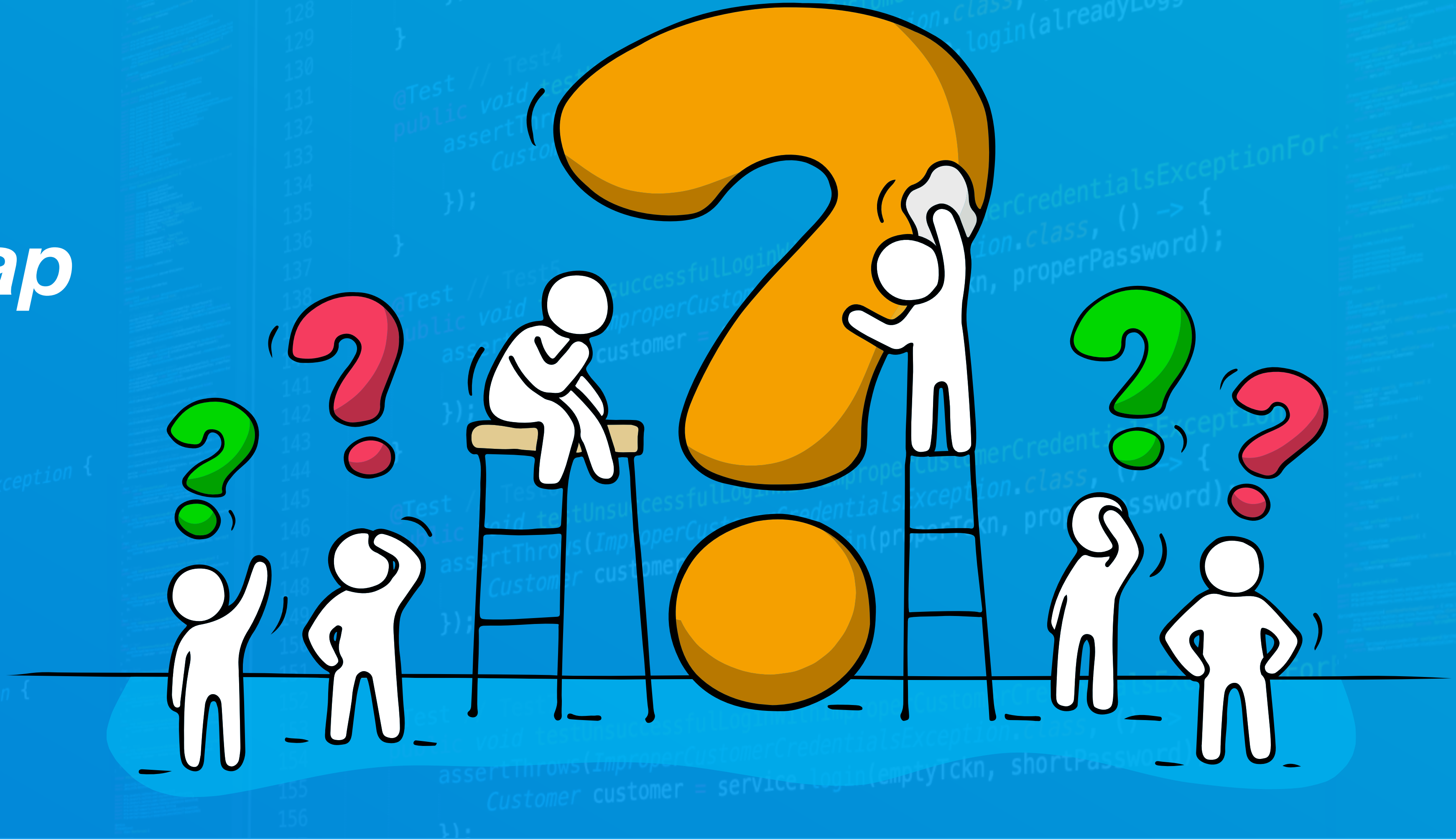
- 5 bir tam sayıdır (integer), `a` bir karakterdir (character), `{1, 2, 3, 4, 5}` bir dizidir, `1`, `Semih`, `Telli`, `Satış`, `true`, bir çalışanı (employee) temsil eden bir nesnedir (object)
- Yani veriler farklı türlerde ya da tiplerde olur
- Aynı **tipten** (**type**) olan veriler aynı karakteristik özelliklere sahiptir, yani, benzerdir
- Tam sayılar, karakterler, doğru-yanlış, çalışan (Employee) nesneleri gibi

# Tip (Type) - II



- Tip, bir veri ya da değer kümesine, özellikleriyle birlikte verilen isimdir
- Tipler, değer kümesinin tabiatına uygun kurallara sahiptir
- Örneğin tam sayılar toplanır ama doğru-yanlış toplanamaz ya da sayılar üzerinde aritmetik işlemler yapılabilir ama metot çağrısı yapılamaz, sadece nesneler üzerine metot çağrısı yapılabilir, örneğin çalışan (employee) nesneleri üzerine metot çağrısı yapıp onlardan çalışmaları (work) istenebilir ama bir karakterden böyle bir şey isteyemeyiz
- Aslında bu ayırım her zaman o kadar keskin olmayabilir de!

# Soru ve Cevap Zamanı!





# Tipli ve Tipsiz Diller

# Tip: Veri ve Değişken



- Verinin tipi var, peki, değişkenin tipi var mıdır?
- Ya da temel soru şudur: Verinin tipi, atandığı değişkeni etkiler mi?
- 5'in tipi tam sayı, **a**'nın tipi karakterdir
- Eğer `i = 5` ya da `i = 'a'` şeklinde bir atama yapılırsa 5 veya **a**'nın tip bilgisi `i`'ye geçer mi?
- Ya da `i`, kendisine atandığı verinin tipiyle uyumlu bir tipe, önceden sahip olmalı mıdır?

# Tipsiz (Typeless) & Tipli (Typed)



- Diller bu sorulara verdikleri cevaplara göre temelde **tipsiz (typeless)** ve **tipli (typed)** olarak ikiye ayrılırlar
- Bu ayırım daha önce derlenen-yorumlanan (compiled-interpreted) ayırımında gördüğümüz gibi programlama dilleri arasında paradigmatik bir ayırımdır

# Veri, Değişken ve Tip



- Python ya da JavaScript gibi dillerde tip, veriye ait bir bilgidir, değişkenin tipi yoktur
- Böyle dillerde değişkenler tuttuğu verinin tipini alır dolayısıyla verinin tipi değiştikçe değişkenin de tipi değişir

```
akin@M3-Max:~/Development/Python/Code
~/Users/akin/Development/Python/Code >python3 variables.py
5
<class 'int'>
True
<class 'bool'>
5.5
<class 'float'>
Selam
<class 'str'>
~/Users/akin/Development/Python/Code >
```

```
# Error!
#int i = 5

# Error!
#float f = 5.5

# Error!
#bool b = True

i = 5
print(i)
print(type(i))

i = True
print(i)
print(type(i))

i = 5.5
print(i)
print(type(i))

i = 'Selam'
print(i)
print(type(i))
```

# Tipsiz ya da Dinamik Tipli Diller



- Bu şekilde, değişkenlerin derleme zamanında tipinin olmadığı ve çalışma zamanında da verisine bağlı olarak belirlendiği ve değişebildiği dillere **dinamik tipli (dynamically-typed)** ya da **tipsiz (typeless)** dil denir
- Python, JavaScript ve Ruby dinamik tipli dillerdendir
- Böyle diller değişkenlerle ilgili herhangi bir tip analizi ya da değişken-veri tipi uyumuyla ilgili bir kontrole de gerek duymazlar

# Tipli ya da Statik Tipli Diller



- Değişkenlerin sabit bir tipinin olduğu ve bu tipin de derleme zamanında belirlendiği ve çalışma zamanında değişmediği dillere **tipli (typed)** ya da **statik tipli (statically-typed)** denir
- Fortran, Pascal, C/C++, C#, Go, Kotlin gibi statik tipli diller, derleme ve çalışma zamanında pek çok tip analizi ve kontrolü yapar
- Zorunlu olmamakla birlikte derlenen (compiled) diller statik tipli, yorumlanan (interpreted) diller dinamik tipli olma eğilimindedir
- Gerçi derlenen-yorumlanan olma daha çok dilin gerçekleştirilmesiyle ilgilidir

# Java Güçlü Tipli Bir Dildir



- Java statik tipli bir dildir dolayısıyla her değişkenin derleme zamanında belirlenen bir tipi olmalıdır ve bu tip bir daha asla değişemez
- Ayrıca Java tiplerle ilgili sıkı kurallara sahiptir
- Bazı statik tipli diller, diğerlerinden daha fazla tip kısıtı ve kuralına sahip olur, bu tür dillere **güçlü-tipli (strongly-typed)** denir
- Örneğin Java'nın C++'a göre daha güçlü tipli bir dil olduğu söylenebilir

# Tip Kontrolleri



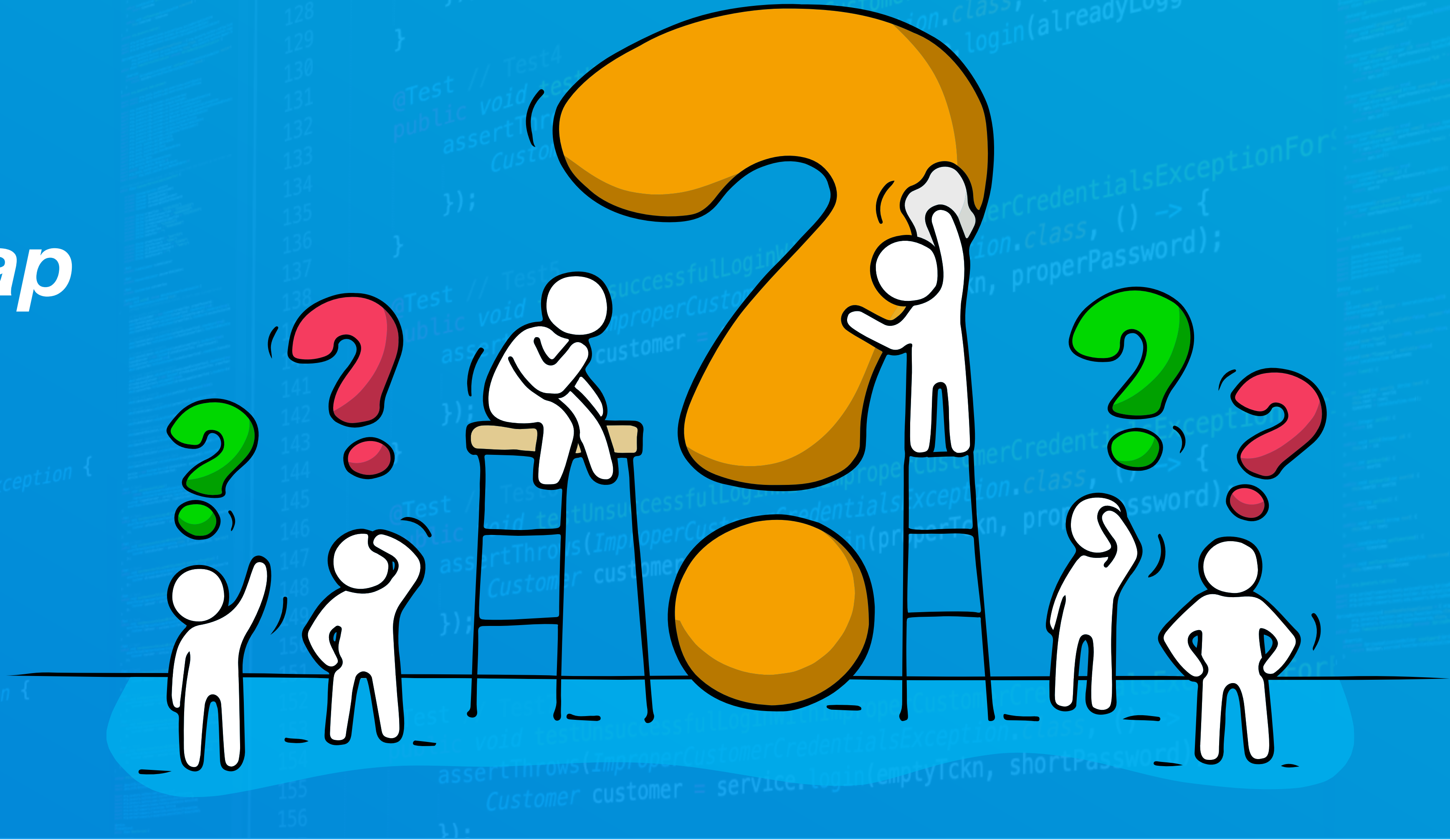
- Java derleyicisi ve JVM, derleme ve çalışma zamanında, tiplerin kural ve kısıtlarını verilere ve değişkenlere uygular:
  - Compile-time type checks
  - Run-time type checks
- Uyumsuzluk durumunda hata üretilir

# Types



- `Types`

# Soru ve Cevap Zamanı!



# Java'nın Tipleri



# Tipler - I



- Aşağıdaki tipler arasında yapıları itibariyle nasıl bir fark vardır?
- `5`
- `a`
- `Selam`
- `3.14`
- `3.141592653589793`
- `true`
- `false`
- `{1, 2, 3, 4, 5}`
- `1, Semih, Telli, Satış, true`

# Tipler - II



- Java'da yapısı itibariyle iki tür tip vardır:
  - **İlkel (basit) tip (primitive type)**
    - Bunlara **ilkeller (primitives)** de denir
  - **Karmaşık tip (complex type)**
    - Bu tiplere, sınıflardan türetildikleri için **sınıf (class)** ya da **nesne (object)** tipleri veya nesnelere referanslarla ulaşıldığı için **referans (reference)** tipleri de denir

# Tipler - III



- Nesne merkezli dillerde, idealde sadece karmaşık tiplerin olması beklenir:
  - `{1, 2, 3, 4, 5}`
  - `1, Semih, Telli, Satış, true`
- Yani her tipin bir sınıf olması ve her değişkenin de bu sınıftan türetilen bir nesneyi göstermesi gerekir



- Çünkü, gerçeklik en temelde nesneler ile anlaşılır ve modellenir, tam sayı ya da karakter gibi ilkel tipler gerçekliği ifade etmede sınırlı kalır
- Evren nesne merkezlidir (object-oriented)
- Bu noktada 5 ya da 3.14 gibi sayıların aslında gerçeklikte olmadığı ve sadece zihnimizin yaptığı soyutlamalar olduğu iddia edilebilir
- Yani aslında var olan 5 elma, 5 insan ya da çemberin çevresinin çapına oranıdır

# Tipler - V



- Bütün tiplerin nesne olduğu yaklaşımdan Java'da pratik nedenlerle bir miktar sapılmış ve ilkel tipler oluşturulmuştur
- İlkel tipler, aritmetik, karakter ve mantıksal işlemleri kolayca yapabilmek için dile konmuştur
- İlkel tipler 8 tanedir ve dilin tasarımı sırasında detayları değişmeyecek şekilde belirlenmiştir ve yenileri de oluşturulamaz

# İlkel Tipler



- İlkel tiplerden olan veriler sadece tek bir değerden oluşur yani atomiktir
- `5`    `a`    `true`    `3.14`
- İlkel tipler tek parçadır ve o parça da değerdir

# Karmaşık Tipler



- Ama karmaşık tiplerin verileri bileşik (composite) yapıdadır, parçaları vardır, birden fazla ilkel ve bileşik tipten oluşur
- `{1, 2, 3, 4, 5}` => 5 tane tam sayı
- `How are you?` => 12 tane karakter içeren metin parçası
- `1, Semih, Telli, Satış, true` => 1 tane tamsayı, 3 tane 5 karakter içeren metin parçası ve bir mantıksal değer

# İlkel ve Karmaşık Tipler - I



- İlkel tiplerin davranışları yoktur, sadece tek bir değeri ifade ederler
- Karmaşık tiplerin, farklı değişkenlerle (variables) gösterilen parçaları olduğu gibi davranışları (behaviors) vardır:
- Metotlar (methods) (fonksiyonlar (functions))

```
public class Selam{  
    String world = "millet";  
  
    public String selamSoyle(String kime) {  
        String cumle;  
        if (kime != "")  
            cumle = "Selam " + kime + " :)";  
        else  
            cumle = "Selam " + world + " :)";  
        return cumle;  
    }  
}
```

# İlkel ve Karmaşık Tipler - II



- Bu yüzden karmaşık tipler yani nesneler veriden çok daha fazlasıdır
- Yani karmaşık tipler, sadece birden fazla verinin bir araya getirilmesi değildir,
- Bir cümle, bir kaç tane kelimenin bir araya getirilmesinden çok daha fazladır
- Fark, anlamdır!

```
public class Selam{  
    String world = "millet";  
  
    public String selamSoyle(String kime) {  
        String cumle;  
        if (kime != "")  
            cumle = "Selam " + kime + " :)";  
        else  
            cumle = "Selam " + world + " :)";  
        return cumle;  
    }  
}
```

# İlkel ve Karmaşık Tipler - II



- İlkel tiplerin Java'da halihazırda tanımlanmış ve değiştirilemez, yenileri de eklenemez olmasına karşın, geliştiriciler, `class`, `enum`, `interface`, `record` vb. anahtar kelimeler kullanılarak yeni karmaşık tipler oluşturur
- Java, tekrar kullanımı (reusability) arttırarak programlamada kolaylık sağlaması amacıyla oluşturulmuş `String`, `Date`, `Scanner`, `File` gibi pek çok hazır karmaşık tipi içeren zengin bir sınıf kütüphanesine (class library) de sahiptir

# İlkel ve Karmaşık Tipler - IV



- Geliştiriciler de uygulamalarında `Customer`, `Account`, `Player`, `FrontController`, `FinancialQueryService`, `Log` gibi yenilerini oluşturur
- Geliştiriciler kendi oluşturdukları tiplerde özellikler ve davranışlar tanımlar:
  - Değişkenler (variables)
  - Metotlar (methods) (fonksiyonlar (functions))



- `hospital` paketi
- Bu örnekteki `Patient` vb. karmaşık tiplerin pek çok ilkel ve karmaşık tipten oluştuğunu ve metotlara sahip olduğunu gözlemleyin

```
public class Patient {
    String id;
    String name;
    char sex;
    int age;
    String[] illnesses;

    public void takeMedication(Prescription prescription){
        // ...
    }

    public void visitedBy(String relative) {
        // ...
    }
}
```

```
public class Doctor {
    String id;
    String name;
    String field;
    Patient[] patients;

    public Prescription inspect(Patient patient){
        Prescription prescription = null;
        // ...
        return prescription;
    }

    public void operate(Patient patient){
        // ...
    }
}
```

# Hospital



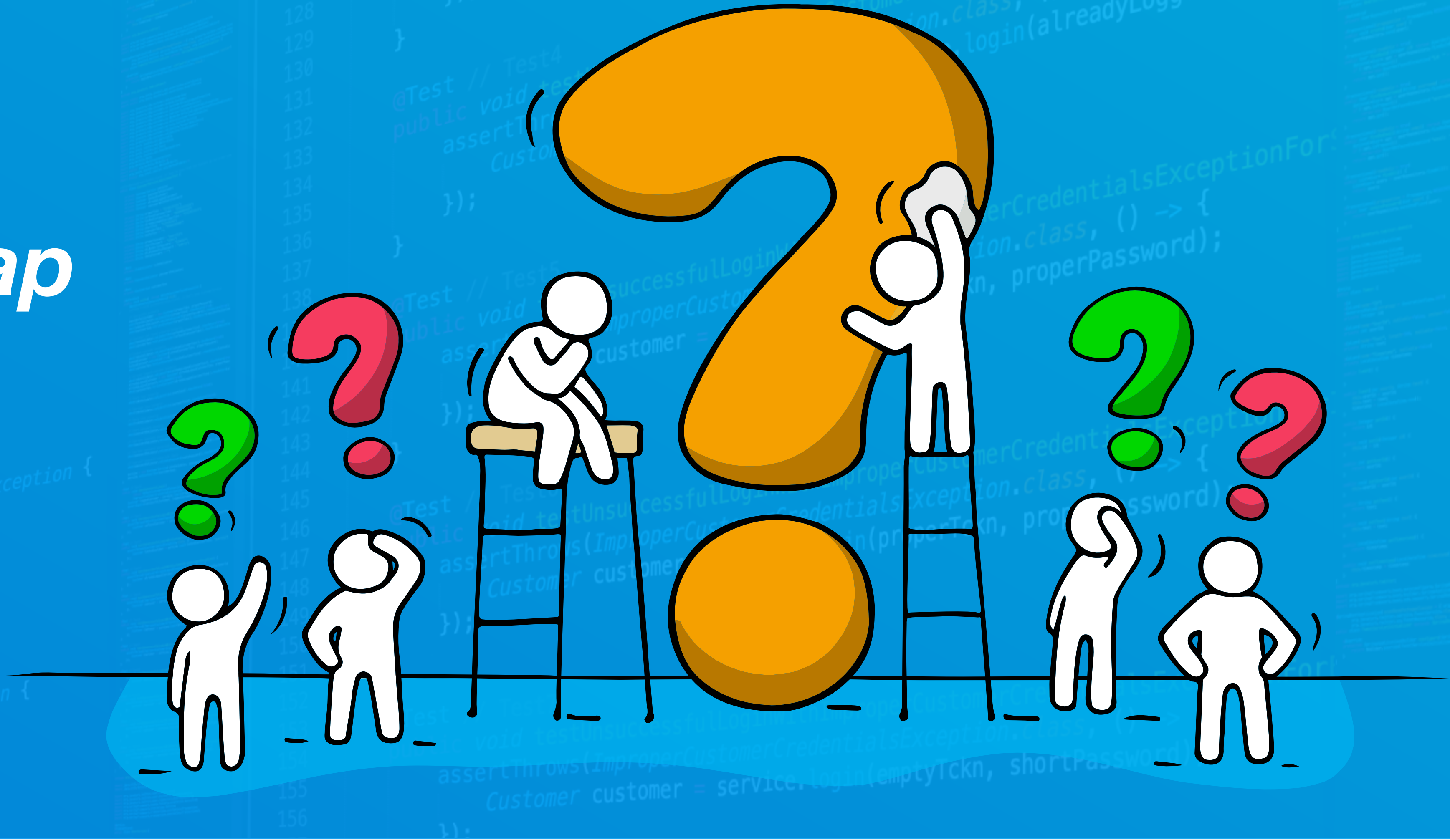
- `hospital` paketi

```
public class Prescription {  
    long id;  
    Doctor doctor;  
    Patient patient;  
    Date date;  
    String[] medications;  
    // ...  
}
```

```
public class Receipt {  
    long id;  
    Date date;  
    double amount;  
    Patient patient;  
    // ...  
}
```

```
public class Hospital {  
    String name;  
    Doctor[] doctors;  
    Patient[] patients;  
    // ...  
  
    public boolean appointmentAt(Date appointmentDate){  
        boolean success = false;  
        // ...  
        return success;  
    }  
  
    public Receipt receive(int payment) {  
        Receipt receipt = null;  
        // ...  
        return receipt;  
    }  
}
```

# Soru ve Cevap Zamanı!



# Ödevler



# Ödev - I



- Aşağıdaki her satırdaki verinin basit mi karmaşık mı olduğunu belirtin
  - 32
  - "Java"
  - false
  - {2, 4, 6, 8}
  - true
  - 7.94
  - 'A'
- ```
class Point{  
    int x, y;  
    double calculateDistanceToOrigin() {  
        ...  
    }  
    ...  
}
```

# Bundan Sonra?



# Bundan Sonra?



- Şimdi sırasıyla önce ilkel tip ve değişkenleri sonra da karmaşık tip ve değişkenleri inceleyeceğiz

# Bölüm Sonu

## Soru ve Cevap Zamanı!

